ZC215: a 215-state ZFC-consistency machine

Exhaustive enumeration, tag lowering, and an opaque initial stack

Research contribution prepared for Vladimir Reshetnikov's ProveIt project

21 October 2026

Abstract

We construct a binary, two-way-tape Turing machine with 215 working states whose blank nonhalting is equivalent to Con(ZFC) by ordinary reasoning formalizable in ZFC. The machine combines DROP-first command tags, a shared CB1 refill, zero-valued spare blocks, and an alternate blank entry. It differs from the accepted `commands120` table in two transition rows and its initial state. A new tag-lowering injection proves that every prefix of the exhaustive command stream contains at least as many DROP operations as NIP operations. Thus the entire initial numerical stack may be arbitrary junk: no initialized reserve of theorem heads is needed, even if every axiom-generated stack entry is ignored for this counting argument. The exact table passes 6,177 arbitrary-value guarded runtime obligations and a complete 117,004-step blank initialization audit. We also give a seven-shift synchronization theorem for a different abstract enumerator, exhibit a concrete false halt in its archived implementation, and record a fresh-binder source experiment. No 214-state machine or new proof-assistant endpoint is claimed.

1 Result and verification boundary

Let M_{ZC215} be the dense transition table in Appendix B, initially in state 0 with its head at cell 0 of an all-zero tape. It has 215 working states, 430 transition rows, alphabet $\{0, 1\}$, and only left and right moves. State 215 is the separate halt state and has no outgoing transitions. Initialization is part of the machine; no input, oracle, or prewritten tape is supplied.

Theorem 1.1. *For the delivered literal table,*

$$\mathrm{ZFC} \vdash \quad \neg\,\mathrm{Halt}(M_{\mathrm{ZC215}}, \mathrm{blank}) \iff \mathrm{Con}(\mathrm{ZFC}).$$

Here consistency means absence of a finite syntactic ZFC refutation, with the proof encoding used by the inherited ProveIt calibration.

The theorem is supported by an ordinary mathematical proof, explicit inherited calibration results, and a fresh executable symbolic derivation of the complete binary simulation. The executable checker uses induction over arbitrary unary lengths and arbitrary finite spare lists; it is not a bounded search over register values. It is also not a proof-assistant kernel. The primary parent K215 has compiled Lean and Rocq endpoints in the repository, but this contribution does not extend those endpoints to the new table. This distinction is part of the release status, not an implicit verification claim.

Quantity	ZC215
Working states / transition rows	215 / 430
Graph-reachable working states	215
Distinct source subtrees / primitive occurrences	202 / 4,752
Maximum source depth / consecutive zero run	47 / 6
Guarded arbitrary-value runtime cases	6,177
Symbolic rule applications in fresh replay	1,698,860
Transition rows covered by the runtime proof	430
Minimum constant term of a runtime duration	9
Literal blank startup transitions	117,004

The source-subtree count is a compression metric, not a count of physical Turing states. Graph reachability likewise does not assert that every state is visited on the particular blank trajectory.

2 The exact construction

2.1 Pinned parent and table changes

The immediate parent is `candidate-k215/alternatives/commands120` at ProveIt revision

`29e2cc4c557c04d3358e41e566bd7c582844ff4e`.

It assigns command tags $\mathsf{DROP} = 0$, $\mathsf{NIP} = 1$, $\mathsf{MP} = 2$, and $\mathsf{AX} \geq 3$. Frozen copies of its native table and source are included in `reference/commands120/`. In the parent's native labels, make exactly the following changes:

Component	Parent	ZC215
Initial state	0	74
Transition at $(8, 1)$	$1\,R\,13$	$1\,R\,103$
Transition at $(198, 0)$	$1\,L\,40$	$0\,L\,40$

The first row change enables the shared CB1 refill. The second changes newly appended spare blocks from $B(1) = 011$ to $B(0) = 01$, where $B(v) = 0\,1^{v+1}$. The native dispatcher is 84 and the native halt label is 254. Native labels are inherited and need not be consecutive. Breadth-first relabeling from state 74, inspecting input 0 before input 1, gives the delivered dense table: start 0, dispatcher 25, halt 215. `machine/canonicalization.json` records the entire bijection.

2.2 The exact source change

The register source is a binary tree of primitive instructions. Its relevant scheduler subtree is

$$((G, (F, (\mathrm{CI0}, H_0))), B), \qquad \begin{cases} G = (\mathrm{CD8}, \mathrm{CB1}), \\ F = (\mathrm{CD0}, (\mathrm{CI8}, (\mathrm{CI6}, \mathrm{CC3}))), \\ H_0 = (\mathrm{CD6}, \mathrm{CC2}). \end{cases}$$

Replace H_0 at tree path `0111` by $(H_0, \mathrm{CB1})$. No other source subtree changes. The new CB1 is primitive occurrence 9, numbered from zero. All old occurrences retain their instructions under the ordinal map $j(i) = i$ for $i < 9$ and $j(i) = i + 1$ for $i \geq 9$. The sole changed old control-flow

edge is the zero branch of old occurrence 7: it now reaches the inserted CB1, which returns to occurrence 0. Every other guarded edge is transported exactly. Two independently implemented control-flow resolvers check the complete graph.

`code/reconstruct.py` derives the source and both presentations of the table from the frozen parent, checks the parent hashes, checks the two guarded graph constructions, and compares the resulting files byte for byte. Thus the construction is a short reproducible delta, rather than an unexplained table found by a search.

2.3 What is materially different

The individual ingredients have repository ancestry: DROP-first command order, CB1 refills, alternate blank entry, and zero spare appending are not claimed as separate first discoveries. Their verified combination is new among the inspected tables. More significantly, their combination changes the proof invariant. The scheduler starts at raw candidate 0 and protects an arbitrary initial stack without decoding even its first head. Section 4 proves this with a tag-order argument that does not use the positive stack contribution of AX.

For comparison, the release preserves all 67 parseable 215-state `.tm` tables present in the pinned checkout, representing 16 rooted transition-graph classes. Canonical comparison found no equality to ZC215 under a rooted state renaming or under that renaming followed by global tape reflection. This establishes a finite, reproducible novelty check within that corpus. It is not a claim about every historical archive or about arbitrary semantic equivalence.

3 Numerical calculus and command contracts

3.1 The inherited consistency calibration

Use Cantor pairing and the numerical constructors

$$P(a,b) = \frac{(a+b)(a+b+1)}{2} + b, \qquad I(a,b) = 4P(a,b), \qquad Q_i(a) = 4P(i,a) + 2.$$

Truth is code 0 and falsity is code 1. In particular $I(0,0) = 0$ is a real alias, not an excluded corner case. Codes are arbitrary natural numbers. The source never checks that a candidate denotes a well-formed first-order formula.

The retained calculus consists of truth, numerical modus ponens, theorem-only generalization, seven logical rows, the gated set-theory seed bundle, and the dominated row

$$L(A,B,C,U) = ((C \to U) \to A) \to ((A \to C) \to (B \to C)), \qquad U \le A,$$

where each arrow is I. The seventh retained row is the merged equality row; the earlier unrestricted eighth row is recovered by the inherited finite derivation from dominated instances. The seed bundle is exactly the parent's bundle, including its gate. No schema, formula substitution rule, or first-order soundness condition is silently added.

We use the parent's calibration theorem: native derivability of code 1 is equivalent, by finite proof transformations formalizable in ZFC, to the existence of a ZFC refutation. The detailed ancestry and numerical definitions are recorded in the pinned repository and the included references [1, 2, 3]. This contribution changes neither that calculus nor its command bodies. It supplies a new scheduler, a new safety proof, and a fresh physical binding.

3.2 The four command equations

Write (t, w, d, h) for top, numerical stack, additive counter, and cached formula. A finite explicit stack $[x_1, \ldots, x_k; w]$ means $P(x_1, P(x_2, \ldots P(x_k, w)\ldots))$. The incoming tail w need not encode any particular list.

$$\mathsf{DROP} : (t, w, d, h) \longmapsto (h, P(t, w), d, h),$$
$$\mathsf{NIP} : \text{halt if } h = 1; \quad \text{otherwise } (t, w, d, h) \longmapsto (t, v, d, x), \quad P(x, v) = w.$$

For $\mathsf{MP}(x)$ put $y = d + x$. If $I(t, y) = h$, return $(y, P(0, w), y, 0)$; otherwise return $(0, w, y, 0)$.

For $\mathsf{AX}(c; x, y)$ put $U = d$, $A = d + x$, $B = h + y$, $C = c$. Its new top is the gated bundle $\mathcal{B}(A, B, c)$; its counter and cache become zero; its new stack is

$$[R_1(A, B, c), \ldots, R_7(A, B, c), L(A, B, c, U), Q_U(h), t; w].$$

The bundle is 0 when $c = A$ or $c = B$. The quantifier entry uses the incoming cache h, before adding y. The dominated side condition is automatic because $A = U + x$.

If top and cache are theorems, DROP pushes a theorem, MP preserves theoremhood and never removes a stack head, and AX pushes ten theorem heads. NIP is safe whenever it has a protected theorem head available, and is the only command that may halt. Its halt test is on the old cache. The important task is therefore to prove that NIP never reads the unprotected initial tail. No interpretation of arbitrary junk codes is needed for that task.

4 Exact enumeration and the tag-lowering theorem

4.1 The refill launches every raw candidate

The nine logical registers are $(N, T, W, A, D, H, X, Y, L)$, with X the refill scratch and L the command residual. At the source root $X = 0$. If $L > 0$, the root decrements it and the reader receives $L - 1$, leaving N unchanged. If $L = 0$ and $N = n > 0$, the two unary transfer loops establish

$$(N, L, X) = (n + 1, n, 0).$$

The inserted CB1 returns to the root; the root then decrements L and the reader receives $n - 1$. The harmless exceptional case $N = L = 0$ performs one additional refill and reads 0 with counter 2. The actual initialized values $N = 1, L = 0$ therefore launch exactly

$$0, 1, 2, 3, \ldots.$$

Each nonhalting candidate finishes. Positive continuations are decremented before their first field is read, and each Cantor projection is at most its input. The next raw continuation is consequently strictly smaller than the current raw candidate. The arithmetic command macros are terminating natural-number loops. Malformed or undecodable formula codes cannot block enumeration.

4.2 Raw codes are a bijection with nonempty command words

The command fields are

$$\mathsf{DROP} : [0], \qquad \mathsf{NIP} : [1], \qquad \mathsf{MP}(x) : [2, x], \qquad \mathsf{AX}(c; x, y) : [c + 3, y, x].$$

For a field list define $E([a_1, \ldots, a_r], v)$ by right-nesting P, with final tail v. For a nonempty command word K, define its raw encoding ρ recursively:

$$\rho([k]) = E(\text{fields}(k), 0), \qquad \rho(k :: K) = E(\text{fields}(k), 1 + \rho(K)).$$

Let $W(n)$ be the syntactic parse of raw candidate n. Unique field counts and strict descent give

$$\rho(W(n)) = n, \qquad W(\rho(K)) = K.$$

These identities include $\rho([\mathsf{DROP}]) = 0$. A raw zero is a one-command candidate; an empty continuation is distinguished by the predecessor's zero residual. A final DROP inside a longer word uses positive residual 1. Losing this distinction would invalidate the argument.

Theorem 4.1 (Tag lowering)**.** *Every finite command prefix of $W(0)W(1)W(2)\cdots$ contains at least as many DROP occurrences as NIP occurrences. Thus assigning balances $+1, -1, 0, 0$ to DROP, NIP, MP, AX gives nonnegative balance at every command boundary.*

Proof. Identify an occurrence by (n, j), the jth command of $W(n)$. For a NIP occurrence, change just that command to DROP, leaving all other commands and payloads unchanged, and let m be the raw encoding of the resulting word. Both commands consume one field, so the new word parses exactly as specified and its jth command is DROP.

Cantor pairing is strictly increasing in either coordinate. Lowering tag 1 to 0 strictly lowers the encoded suffix at the changed command. Every enclosing successor and pairing in the earlier command prefix preserves strict inequality. Hence $m < n$.

The map $(n, j) \mapsto (m, j)$ is injective: from an image DROP occurrence, changing it back to NIP and re-encoding recovers the unique original occurrence. Every NIP in a finite execution prefix therefore has a distinct DROP image in a strictly earlier, already completed candidate. All those images lie in the same prefix. Counting proves the assertion. □

This proof uses no finite testing, diagonal estimate, theoremhood, or AX credit. The earlier independent Cantor-diagonal proof, which assigns AX credit 10, is retained in `proofs/zero_debt_review.md` as a separate corroborating argument. The injection is stronger and shorter.

Proposition 4.2 (Arbitrary-tail safety)**.** *For every $J \in \mathbb{N}$, start the source at*

$$(N, T, W, A, D, H, X, Y, L) = (1, 0, J, 0, 0, 0, 0, 0, 0).$$

Throughout every nonhalting execution, the top, cache, and an explicit finite list of stack heads are numerical theorems; the original tail J is never exposed to NIP. Any halt gives a numerical derivation of 1.

Proof. Initially the explicit list has length zero and top/cache are theorem 0. Maintain a list of theorem heads above the fixed tail J. DROP adds one; NIP removes one; MP never reduces the length; AX adds ten. The actual protected length is therefore at least the conservative DROP-minus-NIP balance. Theorem 4.1, applied to the prefix ending at a NIP, implies that at least one head is available immediately before that pop. It cannot read J. The command equations preserve the invariant. A halt tests a theorem-valued cache equal to 1. □

Explicit list length is a logical witness, not a function of the magnitude of the numerical code. The identity $\text{unpair}(P(x, v)) = (x, v)$ holds even at $x = v = 0$. Thus zero aliases do not undermine the protected-list argument.

The general mechanism is reusable: exhaustive increasing enumeration by a strictly coordinate-monotone, uniquely parsed nested code makes a smaller-tag push pay for a larger-tag pop of the same arity. It gives prefix safety, not merely a favorable average. It also shows why this proof cannot simply be applied to a schedule that skips candidate numbers or starts at an arbitrary positive cutoff.

5 Completeness and composition

5.1 The strong compiler survives arbitrary junk

The inherited command compiler has a stronger interface than a theorem-stack precondition. For every derived code X, it produces a block B_X sending every $(t, w, 0, 0)$ to $(X, w, 0, 0)$, or halting earlier, for arbitrary natural t, w. Its elementary reset is

$$\text{Discard} = \mathsf{MP}(1); \mathsf{AX}(1; 0, 0); \mathsf{NIP}^{10}, \qquad J(0) = \varepsilon, \quad J(u) = \mathsf{MP}(u) \ (u > 0).$$

Against cache 0, $I(t, 1) > 0$ forces MP failure. The following AX has the zero gate $U = A = c = 1$. Its generated entries are implications or quantifiers, hence never code 1; ten pops recover the original stack and cache 0. Omitting $\mathsf{MP}(0)$ in $J(0)$ avoids its possible successful extra push.

From reset values, retained row j is selected by

$$\mathsf{AX}(c; A, B); \mathsf{NIP}^{j}; \mathsf{DROP}; \mathsf{NIP}^{11-j},$$

and the dominated row uses

$$J(U); \mathsf{AX}(c; A - U, B); \mathsf{NIP}^{8}; \mathsf{DROP}; \mathsf{NIP}^{3}.$$

The permitted bundle uses $\mathsf{AX}(c; A, B); \mathsf{NIP}^{10}$. Generalization from a strong block B_X uses

$$B_X; \mathsf{DROP}; J(i); \mathsf{NIP}; \mathsf{AX}(i; 0, 0); \mathsf{NIP}^{9}; \mathsf{DROP}; \mathsf{NIP}; \mathsf{NIP}.$$

For MP from X and $I(X, Y)$, use

$$B_X; \mathsf{DROP}; B_{I(X,Y)}; \mathsf{NIP}; \mathsf{DROP}; \mathsf{NIP}; \mathsf{MP}(Y); \\ \mathsf{AX}(Y; 0, 0); \mathsf{NIP}^{10}; \mathsf{DROP}; \mathsf{NIP}; \mathsf{NIP}.$$

Direct substitution in the equations of Section 3 verifies selection and restoration, including $X = 0$, $Y = 0$, and numerical aliases. Induction on a finite derivation proves the strong compiler. The fuller inherited argument is included in `reference/command-compiler.md`.

If 1 is derivable, append $\mathsf{DROP}; \mathsf{NIP}; \mathsf{NIP}$ to B_1 and prepend a resetting $\mathsf{AX}(0; 1, Y)$. This word halts from arbitrary incoming work registers, either at an earlier test or at its final test of 1. If R is the encoded remaining continuation, its raw candidate number is

$$P(3, P(Y, P(1, R))) \geq Y.$$

Thus successful words occur arbitrarily far out. Our schedule reaches every candidate in order unless a previous semantic halt has already occurred. This proves completeness without requiring any particular interpretation of the starting J.

5.2 The exact blank frame

Starting from the all-zero tape, literal execution of the native table reaches dispatcher 84 after exactly 117,004 transitions, with head -4, header cell 80, gap 20, and source address empty. Its logical registers are

$$(1, 0, 5, 0, 0, 0, 0, 0, 0).$$

The physical block order is $(5, 0, 0, 0, 0, 0, 1, 0, 0)$. The logical-to-physical index map is $(6, 2, 0, 4, 3, 7, 8, 5, 1)$, numbered from zero. There are 67 spare blocks: the third and eighth have value 1, and all others have value 0. Exactly 86 tape cells contain 1; the leftmost is -5 and the rightmost is 240. Every other cell is zero.

The complete list of one-cells is in `evidence/blank_startup.json`. Separate literal simulators reconstruct this entire frame, including the exterior, rather than checking selected registers in an intermediate configuration. Proposition 4.2 applies with $J = 5$. No part of 5 has to be decoded as a safe initial theorem head.

5.3 The arbitrary-value binary binding

For each source occurrence, and separately for both guards of every decrement, the preserved symbolic engine proves a whole-tape binary block. Its domain quantifies over nine independent natural register values, an arbitrary natural gap, and an arbitrary nonempty finite spare list. Symbolic tape atoms represent unary runs and a finite-list tail; literal table steps, unary-loop induction, and separately proved one-block list traversals justify the reductions.

The fresh proof uses the new table and source, with append value 0. All 6,177 cases pass. Each duration is an affine expression in 13 nonnegative slots; all coefficients are nonnegative and the smallest constant term is 9. Together with the generic traversal lemmas, the proof accounts for all 430 transition rows. The release preserves the per-case census and reruns the derivation, comparing every resulting case with that census.

Consequently every nonhalting source step is implemented by a finite, strictly positive number of binary transitions, and the source halt is implemented by the unique binary halt. Positive durations rule out collapsing an infinite source run into a finite binary run. Combining this simulation, the exact blank frame, arbitrary-tail soundness, strong completeness, and the inherited consistency calibration proves Theorem 1.1.

6 A second idea: synchronizing a skipped-candidate schedule

We also studied a substantially different three-command design, with shift S at tag 0, MP at tag 1, and AX at tags at least 2. After S the parser bias is 0; after MP or AX it is 1. A positive continuation L is read as $L - 1 + b$. This design tries to shorten a transfer by allowing a dirty scratch to carry useful bias.

The following is a theorem about the *specified abstract scheduler*. The archived physical scratch-biased table does not implement it; Section 7 gives an exact counterexample.

Let $e(n)$ be 0 when the last syntactic command of raw candidate n is S, and 1 otherwise. With first candidate n_0 and initial bias b_*, the intended launch sequence is

$$n_1 = n_0 + 1 + b_*, \qquad n_{j+1} = n_j + 1 + e(n_{j-1}) \quad (j \geq 1).$$

The delay matters: the next counter was incremented at the beginning of the current candidate. Steps are 1 or 2; cofinality alone does not guarantee visiting any particular successful code.

Lemma 6.1 (Forced hit)**.** *If $k \geq n_0 + 2$, $e(k-2) = e(k-1) = 1$, and $e(k) = 0$, then $k+3$ is launched, unless the machine has already halted.*

Proof. If k is visited, its predecessor is $k-1$ or $k-2$, so the next visit is $k+2$; the ending bit at k then forces $k+3$. If k is skipped, $k-1$ is visited, followed by $k+1$; the ending bit at $k-1$ then forces $k+3$. □

Set $F(x) = P(0, 1 + x)$, which prepends one S. For $x > 0$, exact Cantor decoding gives

$$e(F^7(x) - 5) = 1, \qquad e(F^7(x) - 4) = 1, \qquad e(F^7(x) - 3) = e(x).$$

Thus $e(x) = 0$ makes $F^7(x)$ a forced hit. One way to verify the identities is to use

$$F(x) + 1 = P(x + 2, 0), \quad F(x) - 1 = P(1, x),$$

first at the fourth iterate, and then apply $e(F^3(z) - j) = e(z + 3 - j)$ for $z > 2$ and $j = 3, 4, 5$.

The AX wrapper $A(q) = P(c+2, P(v, P(u, q)))$ preserves all three neighbor ending bits for $q > 5$. Varying v makes the wrapped codes unbounded. A universal refutation word can start with an additional reset and end syntactically with three S commands appended after its guaranteed halt. The appended shifts ensure ending bit 0 and avoid the zero-continuation encoding exception. Seven shifts and an AX wrapper therefore produce a cofinal family of successful codes that this skipped-candidate scheduler is *forced* to visit.

`research/seven_shift_synchronizer.md` supplies the full encoding and boundary argument. A separate all-natural protected-credit proof for this abstract scheduler, with an explicit finite base and asymptotic induction, is in `research/biased_credit_proof.md`; its finite component has a reproducible checker. These are useful specification-level results. They do not certify the rejected table or supply a corrected 215-state allocation.

7 A rejected shortcut and a concrete false halt

The archived `T215_Transfer_2026-10-20` short-transfer experiment inserts CI6 immediately before a CC3 return. Before insertion the CC3 address is `10011`; after insertion it is `100111`. The new address has four one-ancestors. Removing the last three leaves prefix `100`, whose first primitive is ordinal 23, the command dispatch. It does not return to source root 0 as required by the proposed parser equations.

This is a semantic failure even though that archived table passes its own primitive runtime obligations. A valid all-truth frame at MP-entry ordinal 4590, with registers $(1, 0, 0, 0, 0, 0, 0, 0, 0)$, reaches a source halt after 40 primitive transitions. From the corresponding literal binary frame with gap 0 and spare list [0], the table halts after exactly 23,142 binary transitions. A shorter suffix beginning at ordinal 4696 halts after 15 primitive transitions. Complete traces, initial and final one-cells, hashes, and a replay script are included.

This counterexample invalidates the claimed scheduler interface from valid theorem frames. We do not give an unverified numerical time for a false halt from that table's blank start. The affected historical table is included only under `research/rejected_scratch_transfer/`, and is not the ZC215 candidate.

A bounded repair search tested all 1,720 single-row redirects of the relevant `311/1` slot against two proposed corrected return contracts. None passed. Separate local helper searches also failed their global decoder obligations. These are negative results for the stated local searches, not proofs that a corrected 214- or 215-state implementation is impossible. The seven-shift theorem removes a logical obstacle; the physical return remains an engineering obstacle.

8 Formalism-side experiment: flexible fresh binders

Another branch changes the merged equality row itself. Let $E(a, b) = 4P(a, b) + 3$ and choose a primitive-recursive binder function f satisfying

$$a \neq b \implies f(a, b) \notin \{a, b\}.$$

The inherited fresh-binder theorem permits

$$G_f(a, b, c) = E(c, a) \to (E(c, b) \to Q_{f(a,b)} E(a, b))$$

in place of the original binder, with the other rows, gated bundle, numerical MP, and theorem-only GEN retained. It proves consistency equivalence with ZFC, not equality of native theorem sets. Its proof recovers the equality row, interprets equality as indiscernibility in the equality-free seed

calibration, and uses freshness only where required. The full inherited argument is included in `reference/fresh_binder_proof.md`.

We composed the choice $f_Q(a, b) = 4P(a, b) + 2$ with the accepted transfer201 source. Since $P(a, b) \geq a + b$, this binder exceeds both inputs. Replace the final CI1 at path

`1101001110101011000011011`

by $(\mathrm{CI1}, \mathrm{CI1})$. The resulting source has 201 distinct subtrees, 4,776 primitive occurrences, depth 44, and maximum zero run 6. It has 6,210 intended guarded source cases, but no accepted physical runtime certificate.

The extra increment does not increase the number of distinct source subtrees because its pair is already shared elsewhere. This makes it a concrete example of a formalism change that is mathematically larger in one emitter but neutral in the source compression metric. The command selection counts and non-falsity side conditions survive. A corrected restricted allocation model was infeasible; 22 locally valid helper patches all failed complete decoder checks. The delivered `research/transfer_gq.sites.json` is therefore a calibrated source experiment, not an additional binary result.

We also composed four binder families with four neutral source variants each. The resulting compression metrics, common to the four variants within each family, were:

Binder $f(a, b)$	Distinct source subtrees	Primitive occurrences
$4P(a, b) + 2$	201	4,776
$4P(a, b)$	202	4,774
$4P(a, b) + 3$	203	4,777
$a + b + 1$	203	4,729

All four satisfy the required off-diagonal freshness condition. The sum binder saves many primitive occurrences but loses subtree sharing; shortening the arithmetic word alone therefore does not predict a smaller physical machine. The 16 initial allocation attempts all failed in their restricted model, whose first selector protections were inherited from an older allocation policy. The corrected negative result stated above concerns the Q-binder model only. Representative sources and the complete search summary are included; these searches give no general lower bound.

9 Reproduction, provenance, and remaining work

The archive is portable and uses Python's standard library for its main checks. From the extracted directory run

```
python3 -B verify.py --out /tmp/zc215-check
python3 -B verify.py --runtime --out /tmp/zc215-full-check
```

Choose output paths that do not already exist. Assertions must remain enabled; do not use Python's `-O` option. The first command checks all released hashes, exact reconstruction, full source transport, literal startup, saved runtime records, and the finite comparison corpus. The second additionally rederives every arbitrary-value runtime case. It does not merely trust a success flag in the archive.

The tag-lowering proof is in `proofs/tag_lowering_credit.md`; its finite encoding audit is supplementary to the mathematical injection. Research scripts and receipts distinguish abstract

specifications, rejected tables, and successful construction evidence. Build this report with `build_report.sh`; the printed table is generated from the same dense machine and included as `machine_table.tex`.

The main new contribution is a concrete 215-state combination with a stronger zero-reserve proof, not a one-state improvement. Its tag-lowering argument separates stack safety from the amount of axiom output and permits arbitrary initialized junk tails. Future source simplifications can exploit that freedom, but deleting any AX output still requires a revised strong compiler and a fresh binary proof. The seven-shift result gives a second route once its physical return is repaired. The fresh-binder source gives an independent formalism-level degree of freedom.

The most valuable next formalization step is an exact-table Lean or Rocq endpoint for ZC215. The main open compression target remains 214 working states. Neither a minimum-state theorem nor unconditional nonhalting is asserted. The proof establishes the requested equivalence inside ZFC; it does not establish Con(ZFC) within ZFC.

References

[1] V. Reshetnikov et al., ProveIt, accepted K215 construction and command-order alternatives, revision `29e2cc4c557c04d3358e41e566bd7c582844ff4e`. See the pinned ZFC research directory, especially `candidate-k215/README.md`, `proof.md`, and `alternatives/commands120/README.md`. Relevant snapshots are included under `reference/`.

[2] ProveIt, `candidate-k218/alternatives/dominated/proof.md`, same revision. Inherited numerical-calculus and dominated-row consistency calibration. The parent proof records the dependency and the compiled exact-table endpoints for primary K215.

[3] ProveIt, command-permutation ordinary proof, preserved as `reference/command-compiler.md`. This supplies the numerical command equations, strong arbitrary-stack compiler, and treatment of raw zero aliases. Its older physical startup figures refer to its own machines, not ZC215.

[4] ProveIt, Source 165 fresh-binder theorem, preserved as `reference/fresh_binder_proof.md`. Its original inspected revision is retained in that file; the source-side composition in this report uses the pinned current transfer source.

[5] ProveIt historical submission `docs/incoming/T215_Transfer_2026-10-20.zip`, recovered at revision prefix `7571649843`. Only the rejected short-transfer table, source, original explanatory text, and exact counterexample relevant here are included in the research appendix files.

A File identities

All digests below are SHA-256. `MANIFEST.json` covers every released file except itself; generated LaTeX auxiliary files are excluded.

Native table, start 74 / halt 254

`fd9571a0d9bd7a01ab141585e79e956a0a029a608afa8b1ca421e6c4dddbdb47`

Dense table, start 0 / halt 215

`6df5f605cc0d99d4cf51b8425eee4cdfb8997aeef3b84abe6d6f11b7e0dcd083`

Source primitive occurrences

`5f2df4a6c25098d27b20b744b5f8ab3db6ebc8a37e1f01a9abdafc035297563f`

B Complete 215-state transition table

Each entry is "write, direction, next state." The two columns for a state are its actions on reading 0 and 1. H is halt label 215. The blank tape is all 0, the initial state is 0, and the head initially scans cell 0. States 0 through 214 are the 215 working states. There are no omitted working-state rows.

Table 1: The complete ZC215 dense table. $H = 215$ is the halt state.

State	Read 0	Read 1	State	Read 0	Read 1
0	1R1	1R2	108	0R125	1R126
1	1R3	1R1	109	0R127	1R128
2	0R0	1R2	110	0R50	1R129
3	0R4	0R1	111	0R50	1R130
4	1R5	1L6	112	0R131	1R132
5	0L7	1L8	113	0R133	0L10
6	0L7	1L6	114	0R59	1R81
7	0L9	1L6	115	0R80	1R101
8	0L10	0L11	116	0L103	1R134
9	1R12	0L9	117	0R96	1L119
10	0L9	0L10	118	0R135	1R136
11	1L8	1L11	119	0L30	1R97
12	0R12	0R13	120	0R137	1R138
13	0L14	0R13	121	0R139	1R140
14	1L15	0L15	122	0R141	1R127
15	0L15	1L16	123	0R100	1R100
16	0L17	1L16	124	0R142	1R143
17	0L18	1L16	125	0R111	1R110
18	0L19	1L16	126	0R140	1R123
19	0L20	1L16	127	0R126	1R140
20	0L21	1L16	128	0R70	1R100
21	0L22	1L16	129	0R144	1R36
22	0L23	1L16	130	0R56	1R145
23	0L24	1L16	131	0R105	1R146
24	0R24	1R25	132	0R147	1R148
25	0R26	1R27	133	0R120	1R149
26	0R28	1R29	134	0R103	1L42
27	0R30	1R31	135	0R121	1R64
28	0R32	0L7	136	0L150	1R111
29	0R33	1R34	137	0R151	1R46
30	0R35	1R36	138	0R149	1R43
31	0R37	1R38	139	0R35	1R152
32	0L39	0R40	140	0R153	1R154
33	0R41	1R42	141	0R155	1R156
34	0R41	1R43	142	0R157	1R158
35	0R44	1R45	143	0L159	1R160
36	0R46	1R47	144	0R48	1L161
37	0R48	1R49	145	0R46	1R162
38	0R50	1R51	146	0R140	1R163
39	0L52	1R53	147	0R143	1R150
40	0R54	1R40	148	0R114	1R56
41	0L39	0R55	149	0L161	0L15
42	0R32	1L56	150	0R164	1R133
43	0R57	0L10	151	0R84	1R43
44	0R32	1R58	152	0R145	1R111
45	0R59	1R42	153	0R117	1R99
46	0R60	0R61	154	0R98	1R165
47	0R62	0L14	155	0R111	1R38
48	0R60	0R63	156	0R140	1L109
49	0R52	0L10	157	0R38	1R91
50	0R64	1R65	158	0R166	1R167
51	0R66	1R67	159	0R126	1R168
52	0R68	1R69	160	0R38	1R110

ZC215 transition table (continued)

State	Read 0	Read 1	State	Read 0	Read 1
53	0R70	1R71	161	0R66	1R115
54	0R14	1R54	162	0R116	0L14
55	0R72	1R55	163	0R121	1R125
56	0R73	1L51	164	0R156	1R169
57	0R46	0L13	165	0R59	1R170
58	0R74	1R75	166	0R92	1R156
59	0L64	1L58	167	0R171	1R156
60	0L76	0R1	168	0R136	1R172
61	0R77	1R61	169	0R173	1R174
62	0R78	0R63	170	0R175	0L14
63	0R79	1R63	171	0R110	1R38
64	0R80	1R81	172	0R108	1R176
65	0R82	1R83	173	0R140	1R177
66	0R84	1R85	174	0R178	1R179
67	0R46	1R86	175	0R123	1R123
68	0R48	1R87	176	0R180	1R110
69	0R88	1R89	177	0R166	1R181
70	0L90	1L91	178	0R182	1R183
71	0R92	1R93	179	0R184	1R185
72	0R63	1R72	180	0R90	1R183
73	0R60	0R79	181	0R123	1R167
74	0R78	0R72	182	0R186	1R187
75	0R94	0L81	183	0L188	1R189
76	1R76	1L15	184	0L190	1L91
77	0R55	1R77	185	0R183	1R144
78	0L76	0R2	186	0R191	1R159
79	0R95	1R79	187	0R190	1R192
80	0R60	0R95	188	0R184	1R193
81	0R88	0L14	189	0R90	1R140
82	0R96	1R97	190	0R173	1R90
83	0R98	1R99	191	0R123	1R184
84	0R60	0R77	192	0R127	1R194
85	0R100	1R59	193	0R184	1R195
86	0R101	0L14	194	0R196	1R188
87	0R102	0L10	195	0R197	1R198
88	0L103	1L67	196	0R199	1R200
89	0R104	1R105	197	0R189	1R183
90	0L106	1L107	198	0R201	1R197
91	0R108	1R109	199	0R202	1R180
92	0R110	1R111	200	0R141	1R203
93	0R112	1R113	201	0R184	1R204
94	0R57	1R114	202	0R107	1R205
95	0R40	1R95	203	0R206	1R207
96	0L39	0R54	204	0R208	1R208
97	0R100	0L14	205	0R184	1R209
98	0R115	1R116	206	0R106	1R210
99	0R59	1R117	207	0R211	1R198
100	0R78	0R95	208	0R160	1L212
101	0L44	1L66	209	0R70	1R213
102	0R39	1R118	210	0R171	1R208
103	0R74	1R119	211	0R125	1R214
104	0R120	1RH	212	0R143	1R166
105	0L121	1R67	213	0R108	1R70
106	0R110	1R122	214	0R125	1R158
107	0R123	1R124			